AI REVENUE RECOVERY AGENT

Razorpay Buildathon — Track 03

Complete Project Plan, Architecture, Technology Stack, AI Strategy, Development Roadmap & Free-Resource Plan

Build a bounded AI agent that detects revenue at risk, explains why it is at risk, selects the safest recovery intervention, and executes or escalates the action.

Project target	₹0 planned development cost
Primary AI provider	Groq
Fallback providers	Gemini → Cerebras → Hugging Face → Cohere
Agent framework	LangGraph
Backend	Python + FastAPI
Frontend	React + Vite
Initial database	SQLite
Payment platform	Razorpay APIs / webhooks

Prepared as the master blueprint for step-by-step implementation.

Contents

1. Executive Summary

The AI Revenue Recovery Agent is an intelligent revenue-recovery control center for merchants. It identifies payment failures, checkout abandonment and overdue receivables; estimates the amount and likelihood of recovery; uses AI to reason about the situation; validates the proposed action against deterministic business rules; and then executes a bounded action or sends the case for human review.

The central design principle is: the LLM recommends; deterministic rules control execution. This keeps financial actions predictable, auditable and safe while still using AI for reasoning, classification, explanations and personalized recovery messaging.

The system is designed for a free-first build. It will avoid paid model dependencies and support multiple hosted LLM providers so that one provider being unavailable or rate-limited does not stop the application.

2. Problem Statement

Revenue leakage often occurs after a customer has already shown intent to pay. A failed payment may never be retried, an abandoned checkout may be forgotten, or an overdue invoice may remain unpaid. A conventional dashboard reports these events but leaves the merchant to manually investigate and decide what to do.

```
Customer starts checkout
↓
Payment fails / checkout is abandoned / invoice becomes overdue
↓
Merchant must investigate manually
↓
Follow-up may be delayed
↓
Recoverable revenue becomes lost revenue
```

Our product changes this from passive reporting to an active recovery workflow.

3. Product Vision

The final product should feel like a Revenue Recovery Control Center. The merchant should be able to answer five questions immediately:

- What is at risk? Identify affected transactions, customers and receivables.
- Why is it at risk? Explain the contributing signals in plain language.
- How much can we recover? Estimate the value and prioritize cases.
- What should we do? Recommend retry, reminder, checkout recovery or escalation.
- Can we safely automate it? Validate the action using deterministic policy before execution.

4. Core Use Cases

4.1 Failed Payment

Detect a failed payment, inspect the customer's payment history and failure characteristics, calculate risk, and decide whether an automatic retry, reminder or escalation is appropriate.

4.2 Checkout Abandonment

Detect a checkout session where the customer showed purchase intent but did not complete payment. The system can prioritize recent, valuable abandoned sessions and generate a recovery message.

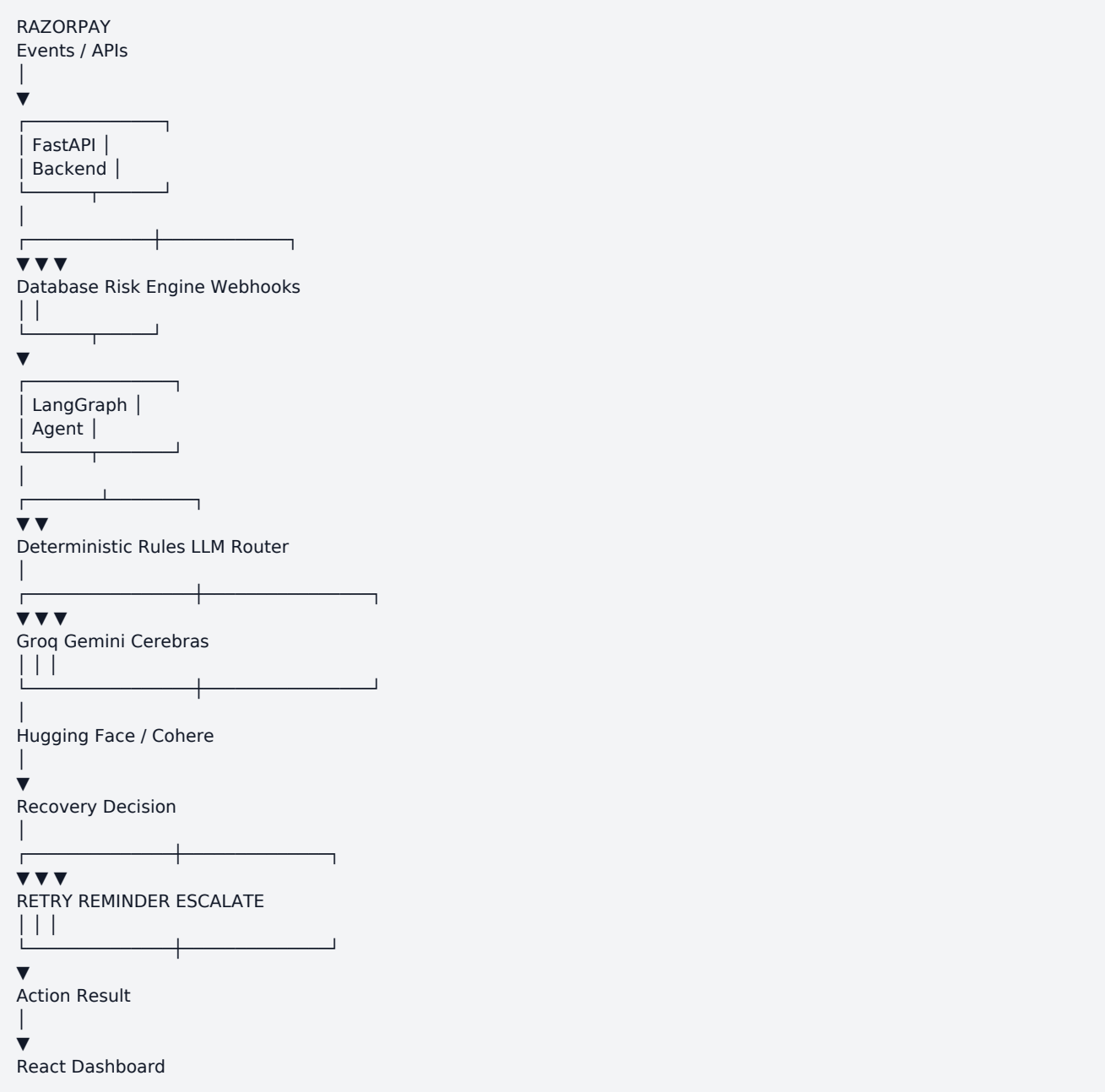
4.3 Overdue Receivable

Identify unpaid invoices after their due date. The agent can recommend a reminder for normal cases and manual review for large or sensitive receivables.

4.4 High-Value Risk

High-value transactions can be routed to manual approval even when the AI recommendation is otherwise valid.

5. System Architecture



6. AI + Deterministic Design

The system intentionally separates reasoning from authorization. The LLM can analyze context, classify a case, generate an explanation and propose an action. A deterministic rules layer decides whether that action is permitted.

```
LLM:
"Recommend RETRY because the failure appears temporary."

Rules:
```

```
Is retry allowed?
Has retry limit been reached?
Is transaction within automatic-action threshold?
Is the event valid?
Does this action require approval?

Only if policy passes:
→ execute
```

This architecture also makes the system easier to explain to judges and safer to evolve.

7. Multi-Provider LLM Strategy

We will not hard-code the application to a single AI provider. An LLM Router will provide a common interface to the agent and maintain an ordered fallback chain.

```
Primary:
1. Groq

Fallbacks:
2. Google Gemini
3. Cerebras
4. Hugging Face
5. Cohere
```

Example failure path:

```
Request → Groq → unavailable/rate-limited
→ Gemini → unavailable
→ Cerebras → success
→ return structured result to LangGraph
```

The application will use only providers/models that are currently available under suitable free or trial access. Free-tier quotas, model availability and terms can change, so they should be verified at setup time.

Provider configuration

```
GROQ_API_KEY=
GEMINI_API_KEY=
CEREBRAS_API_KEY=
HUGGINGFACE_API_KEY=
COHERE_API_KEY=
```

Only the credentials for providers we actually enable need to be populated. Secrets must remain in .env and never be committed to Git.

8. Technology Stack

Layer	Technology	Purpose
Primary language	Python	Backend, AI, rules, integrations
Frontend language	JavaScript / TypeScript	Dashboard
Backend	FastAPI	REST APIs and webhooks
Agent	LangGraph	Stateful AI workflow
LLM abstraction	LangChain-compatible interfaces	Provider integration
Primary LLM	Groq	Fast hosted inference
Fallback LLMs	Gemini, Cerebras, Hugging Face, Cohere	Resilience

Layer	Technology	Purpose
Database	SQLite initially	Local development
ORM	SQLAlchemy	Database access
Validation	Pydantic	Typed request/response models
Frontend	React	Merchant dashboard
Build tool	Vite	Frontend development/build
Payments	Razorpay APIs/webhooks	Payment events and actions
Version control	Git + GitHub	Source control

9. Razorpay Integration

Razorpay events will feed the revenue-risk pipeline. During development and demonstration, use test/sandbox credentials and test data wherever applicable.

```
Razorpay event
↓
Webhook endpoint
↓
Validate event
↓
Normalize event
↓
Store transaction state
↓
Run risk detection
↓
Create/update recovery case
↓
Trigger agent when appropriate
```

The exact webhook event names and API operations should be implemented from Razorpay's current documentation and test environment rather than assumed from examples.

10. Data Model

10.1 customers

```
id
name
email
phone
created_at
total_payments
successful_payments
failed_payments
```

10.2 payments

```
id
customer_id
amount
currency
status
failure_reason
created_at
updated_at
```

10.3 checkouts

```
id
customer_id
amount
status
started_at
abandoned_at
```

10.4 invoices

```
id
customer_id
amount
due_date
status
paid_at
```

10.5 recovery_cases

```
id
customer_id
payment_id
case_type
risk_score
reason
recommended_action
status
created_at
```

10.6 recovery_actions

```
id
case_id
action_type
executed_at
result
message
```

11. Backend API

Endpoint	Method	Purpose
/health	GET	Health check
/customers	GET	Customer list
/payments	GET	Payment list
/recovery-cases	GET	Revenue-risk cases
/recovery-cases/{id}	GET	Case details
/recovery-cases/{id}/analyze	POST	Run agent analysis
/recovery-cases/{id}/execute	POST	Execute approved action
/webhooks/razorpay	POST	Receive payment events
/analytics	GET	Dashboard metrics

12. LangGraph Agent Workflow

```
START
↓
Load recovery case
↓
Gather customer + transaction context
↓
```

```
Calculate deterministic risk signals
↓
LLM analysis/classification
↓
Generate recommendation + explanation
↓
Policy / safety validation
↓
┌──────────────┐
│ allowed? │
└──────┬───────┘
YES │ NO
↓ ↓
Execute Escalate
↓ ↓
Record outcome
↓
END
```

Agent state should include customer ID, transaction ID, amount, failure reason, customer history, risk score, reasoning summary, recommended action, policy result and execution result.

13. Risk Scoring

The first version should use an interpretable deterministic score rather than requiring a trained machine-learning model. This makes the demo reproducible and easy to debug.

Signal	Example contribution
Transaction value	Higher value → higher priority
Repeated failures	More failures → higher risk
Customer history	Strong prior payment history can increase recovery potential
Failure type	Temporary/recoverable signals can increase recovery likelihood
Days overdue	Older receivables receive higher priority
Checkout age	Recent abandonment can be more actionable
Previous recovery attempts	Repeated unsuccessful attempts reduce automation eligibility

```
Example:
Transaction value +20
Repeated failure +20
Strong customer history +15
Recent failure +15
Recoverable failure +12
----
Risk score 82/100
```

The exact weights and thresholds will be tuned using synthetic test cases.

14. Recovery Actions

14.1 Retry

Used for eligible payment failures that pass the retry policy.

14.2 Reminder

Used for overdue invoices or eligible failed payments.

14.3 Checkout Recovery

Generate a concise recovery message for a recently abandoned checkout.

14.4 Escalation

Route high-value, repeated-failure or policy-sensitive cases to human review.

15. Safety and Human-in-the-Loop

Financial workflows need bounded automation. The system should include allowlists, thresholds, retry limits, validation and audit logging.

```
Low-risk case
→ automatic action if policy passes

Medium-risk case
→ AI recommendation + merchant approval

High-value / sensitive case
→ manual review
```

- Never let the LLM directly bypass policy.
- Apply maximum retry counts.
- Apply transaction-value thresholds.
- Validate input before execution.
- Keep an audit record of recommendations and actions.
- Escalate ambiguous or unsupported actions.

16. Frontend Dashboard

The dashboard should present the system as a merchant control center rather than an AI chat interface.

16.1 Overview

```
AI REVENUE RECOVERY

Revenue At Risk ₹2,43,500
Recoverable ₹1,71,000
Recovered ₹94,500
Recovery Rate 54.2%

HIGH PRIORITY CASES
Rahul ₹12,000 92 RETRY
Priya ₹8,500 81 REMINDER
Amit ₹51,000 95 REVIEW
```

16.2 Case Detail

```
Customer: Rahul
Transaction: ₹12,000
Risk Score: 92/100

Why at risk?
• Payment failed twice
• Strong previous payment history
• Failure appears temporary

Recommendation: RETRY
Policy check: PASSED
[Execute Recovery]
```

16.3 AI Provider Status

A later dashboard version can display provider health, current provider, fallback count and response latency. This is useful for demonstrating the multi-provider architecture.

17. Project Folder Structure

```
ai-revenue-recovery/
│
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── config.py
│   │   ├── api/
│   │   │   ├── payments.py
│   │   │   ├── recovery.py
│   │   │   └── webhooks.py
│   │   ├── agent/
│   │   │   ├── graph.py
│   │   │   ├── nodes.py
│   │   │   └── state.py
│   │   ├── rules/
│   │   │   └── recovery_rules.py
│   │   ├── services/
│   │   │   ├── risk_service.py
│   │   │   ├── recovery_service.py
│   │   │   └── razorpay_service.py
│   │   ├── models/
│   │   │   └── models.py
│   │   └── database/
│   │   └── database.py
│   ├── tests/
│   ├── requirements.txt
│   └── .env
│
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/
│   │   └── App.jsx
│   └── package.json
│
├── data/
│   └── seed_data.json
├── docs/
│   └── architecture.md
├── .gitignore
├── README.md
└── LICENSE
```

18. Free Resources and Cost Strategy

The project will be developed with a ₹0 target wherever free/open-source options are sufficient. Hosted AI providers can impose changing free-tier limits, so the design deliberately supports multiple providers.

Resource	Planned use	Cost target
Python	Backend + AI	Free
FastAPI	Backend API	Free
LangGraph	Agent orchestration	Free/open-source
React	Frontend	Free

Resource	Planned use	Cost target
Vite	Frontend tooling	Free
SQLite	Initial database	Free
SQLAlchemy	ORM	Free/open-source
Git	Version control	Free
GitHub	Repository	Free tier
VS Code	Development	Free
Groq	Primary hosted LLM	Free/trial access where available
Gemini	Fallback LLM	Free/trial access where available
Cerebras	Fallback LLM	Free/trial access where available
Hugging Face	Fallback/open models	Free options where available
Cohere	Fallback LLM	Free/trial access where available
Razorpay test tools	Payment integration testing	Use test/sandbox facilities

No local Ollama dependency is planned. The architecture uses hosted inference providers instead.

19. Development Roadmap

Phase	Deliverable
Phase 1 — Project Setup	Create backend/frontend structure, Python environment, dependencies, .env and .gitignore. Verify FastAPI starts.
Phase 2 — Database	Create SQLite schema and synthetic customers, payments, checkouts and invoices.
Phase 3 — Risk Engine	Implement deterministic detection and interpretable risk scoring.
Phase 4 — LangGraph	Build the stateful agent graph and test it using synthetic cases.
Phase 5 — LLM Router	Integrate Groq first, then add Gemini, Cerebras, Hugging Face and Cohere fallback adapters.
Phase 6 — Recovery Actions	Implement retry simulation, reminders, checkout recovery and escalation.
Phase 7 — Razorpay	Connect test/sandbox events and relevant API operations.
Phase 8 — React Dashboard	Build overview, case detail, analytics and action screens.
Phase 9 — Testing	Test edge cases, provider failures, policy failures and action outcomes.
Phase 10 — Deployment & Demo	Deploy using suitable free tiers, prepare seed data, polish UI and rehearse the demo.

20. Testing Strategy

Synthetic data is important because it lets us demonstrate the complete system without depending on live customer activity.

- One temporary payment failure after many successful payments → recommend retry.
- Repeated payment failures → reminder or escalation depending on policy.
- Recent checkout abandonment → checkout recovery.
- Normal overdue invoice → reminder.
- Very large overdue invoice → manual review.
- Retry limit reached → block automatic retry.
- Provider timeout → invoke LLM fallback.
- All providers unavailable → fail safely and surface an understandable error.
- Invalid webhook/input → reject and log.
- Successful recovery → update recovery metrics.

21. Demo Story

The strongest demo should be a short end-to-end story rather than a tour of every feature.

1. Dashboard opens with revenue at risk.
2. Select a high-priority failed payment.
3. Show the risk score and contributing signals.
4. Run the AI analysis.
5. AI recommends RETRY and explains why.
6. Deterministic policy validates the action.
7. Execute the bounded recovery action.
8. Show success/failure result.
9. Dashboard updates recovered revenue.
10. Optionally demonstrate Groq failure and automatic fallback to another provider.

The key message to judges is: we don't just detect lost revenue; we identify recoverable opportunities, choose an intervention and safely execute the workflow.

22. Stretch Features

- Recovery probability prediction.
- Customer segmentation such as VIP, regular, at-risk and inactive.
- Smart timing for reminders.
- A/B testing of recovery messages.
- Historical recovery analytics.
- Provider performance analytics.
- Learning loop using previous recovery outcomes.
- Merchant-configurable automation thresholds.
- More sophisticated fraud/risk safeguards.

23. Security

- Store API keys only in environment variables.
- Never commit .env to GitHub.
- Use test/sandbox credentials during development.

- Validate webhook payloads according to current Razorpay guidance.
- Validate API input with Pydantic.
- Keep recovery actions behind explicit policy checks.
- Record action history for auditability.
- Use least-privilege credentials where supported.
- Avoid placing sensitive customer/payment information in LLM prompts unless necessary and permitted.

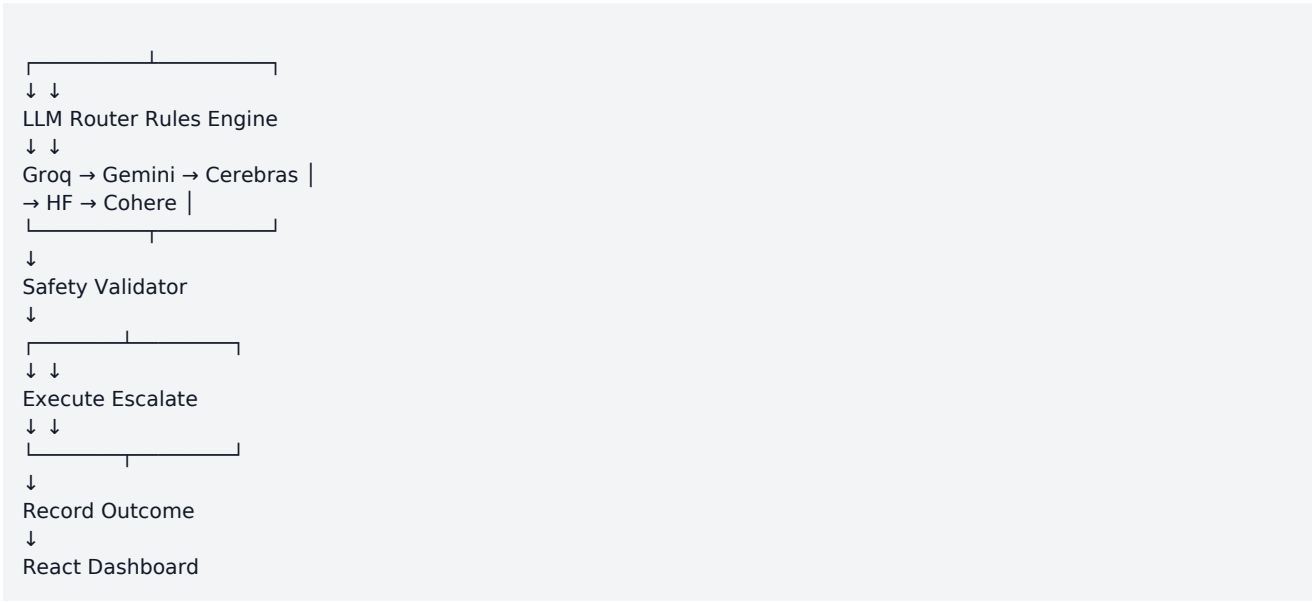
24. Final MVP Checklist

Area	MVP requirement	Status
Backend	FastAPI server	To build
Database	SQLite schema + seed data	To build
Risk	Deterministic risk scoring	To build
Agent	LangGraph workflow	To build
LLM	Groq primary	To build
Fallbacks	Gemini, Cerebras, Hugging Face, Cohere	To build
Actions	Retry/reminder/escalation	To build
Razorpay	Test/webhook integration	To build
Frontend	React dashboard	To build
Analytics	At-risk/recovered metrics	To build
Safety	Rules + human approval	To build
Testing	Synthetic scenarios + failure tests	To build
Deployment	Free-tier target	To build
Demo	End-to-end recovery story	To build

25. Final Architecture Summary

The final system is a FastAPI-based revenue intelligence backend connected to Razorpay events and a SQLite/PostgreSQL-compatible data layer. LangGraph orchestrates the stateful agent. An LLM Router provides resilient access to Groq, Gemini, Cerebras, Hugging Face and Cohere. Deterministic policies sit alongside the AI layer and are responsible for authorizing actions. React provides the merchant-facing dashboard.

```
REVENUE EVENT
↓
FastAPI
↓
Event Normalization
↓
Risk Detection
↓
Risk Score + Context
↓
LangGraph Agent
↓
```



Project principle: Working first, AI second, automation third, Razorpay integration fourth, dashboard polish fifth.

Target: A credible, explainable, resilient and demo-ready AI revenue recovery system built with free/open-source tooling and free-tier hosted AI services where available.